Member-only story

Boris Cherny's Claude Code Tips Are Now a Skill. Here Is What the Complete Collection Reveals.

The complete 42-tip collection is now an installable Claude Code skill. Parts 3–5 — almost entirely uncovered — reveal the design philosophy behind the full stack.

9 min read · Mar 22, 2026

Reza Rezvani Follow

The first time I read Boris Cherny's January thread, I did what most developers did. I starred the tweet, implemented two tips, and moved on. The CLAUDE.md thing. The parallel sessions thing. Classic.

Boris Cherny's 42 Tips as a SKILL.md File | Image Generated with Gemini Pro ©

Note: *AI tools assisted with researching this piece. The testing, production context, and analysis are from my direct experience running Claude Code daily.*

Then last week I loaded his tips as a Claude Code skill — all 42 of them, across five threads, from January through the end of February 2026 — and something shifted. Not because the tips were new. Because reading them as a system, rather than as individual threads, revealed a design philosophy that the Twitter format completely obscures.

Boris did not share 42 tips. He shared one workflow, incrementally. And when you see it assembled, the parts he released last — Parts 3 through 5, which almost nobody has written about — make the whole thing click into place.

The Skill, First

The `boris` skill was compiled by @CarolinaCherry from Boris's five threads and lives at `~/.claude/skills/boris/SKILL.md`. Install it:

```
mkdir -p ~/.claude/skills/boris
curl -L -o ~/.claude/skills/boris/SKILL.md \
  https://howborisusesclaudecode.com/api/install
```

Then in any Claude Code session: `/skills boris` loads the complete collection.

What you get: 42 tips across 30 topics. Parallel execution, model selection, plan mode, CLAUDE.md, skills and slash commands, subagents, hooks, permissions, MCP integrations, long-running tasks, verification — and then the parts most developers missed entirely.

I have been running this skill for a week across our team of 7. This article is about what the full collection reveals that the January coverage missed.

What January Covered (And Why It Was Incomplete)

When Boris's first thread dropped on January 2, 2026, it went viral for obvious reasons. 7.9 million views. The creator of Claude Code sharing his actual setup.

The coverage focused on three things:

Parallel sessions. Run 5 to 10 Claude instances simultaneously. Boris keeps 5 in his terminal and 5 to 10 more on claude.ai/code. Yours does not have to be that aggressive, but the principle holds: Claude Code compounds when you parallelize.

Plan mode. Shift+tab to start in plan mode, refine until the plan is solid, then switch to auto-accept edits. Claude 1-shots the implementation when the plan is right.

CLAUDE.md. After every correction: *"Update your CLAUDE.md so you don't make that mistake again."* Boris said it is *"eerily good at writing rules for itself."* He was underselling it.

These three tips dominated every write-up, including my own. And they are correct. I implemented them across our codebase in January and the productivity shift was real. Code review cycles went from two to three days to under four hours. The morning brief stopped saying *"pnpm not npm"* for the first time in months.

But that is not the whole story.

What Parts 3 Through 5 Actually Say

The second thread *(January 31)* covered team patterns and went deeper on parallel work. Good, but also widely covered.

Then Boris dropped Parts 3, 4, and 5 across February — and these are the ones I have seen almost nobody write about seriously.

The Customization Layer (Part 3)

Part 3 arrived February 11, 2026. Twelve tips covering terminal configuration, effort levels, plugins, custom agents, permissions management, sandboxing, status lines, keybindings, hooks, spinner verbs, output styles.

The reaction I have seen: developers skimmed it. Spinner verbs? Output styles? Feels cosmetic.

It is not cosmetic.

Output styles change how Claude thinks, not just how it writes. Set the `Explanatory` output style when onboarding someone new to a codebase and Claude explains *why* it is making changes as it works. Set `Learning` and it coaches you through the change rather than making it for you. These are not tone tweaks. They are different cognitive modes.

Our team now uses `Explanatory` by default for any junior engineer working in an unfamiliar service. The PR review time for those changes dropped because the code arrives with reasoning already attached. The reviewer is reviewing a decision, not reconstructing one.

Custom agents change what Claude assumes about its role. Drop a `.md` file in `.claude/agents/` with a custom name, color, tool set, and permissions model. Boris uses `code-simplifier` and `verify-app` as standard subagents. We built a `backend-reviewer` that pre-loads our API standards and a `migration-guard` that has read-only access to our schema history.

The key detail most developers miss: you can set a *default* agent for the main conversation. `"agent"` in `settings.json` or `--agent` at launch. Your main Claude Code session is now running as a specific, constrained, configured entity rather than a blank slate.

Sandboxing is not just for security. `/sandbox` enables file and network isolation for BashTool. Yes, this reduces the blast radius of mistakes. But the more practical effect for our team: it removed the cognitive overhead of wondering whether Claude might touch something it should not. When you trust the containment, you accept edits faster. That speeds up the whole loop.

Worktree Native Support (Part 4)

Part 4 arrived February 20. Five tips covering Claude Code's built-in worktree support.

The `--worktree` flag is the feature Boris has been building toward since January. When he originally shared his parallel setup, he was using separate git checkouts

because worktrees had limitations. The native support changes the math:

```
claude --worktree my_feature  # Claude gets its own worktree
claude --worktree my_feature --tmux  # Plus its own Tmux session
```

More practically: subagents can now use worktree isolation. This is the enabling capability for the migration prompt Boris shared:

> "Migrate all sync io to async. Batch up the changes, and launch 10 parallel agents with worktree isolation. Make sure each agent tests its changes end to end, then have it put up a PR."

Ten parallel agents, each isolated, each testing, each opening a PR. This is not an advanced trick. It is a standard workflow once you have native worktree support. We have been running parallel migrations on our test suite reorganization this week using exactly this pattern.

/simplify and /batch (Part 5)

Part 5 was February 27. Two tips. The densest release.

`/simplify` runs parallel agents that review changed code for reuse opportunities, quality problems, and efficiency issues — all in one pass. Boris uses it daily, appended to his normal prompts:

```
> hey claude make this code change then run /simplify
```

I have been using this for five days. The honest assessment: it catches things I catch in code review. Not everything. Roughly the class of issues that a senior engineer would flag in the first five minutes of a review: duplicate logic that could be a shared utility, an overly nested conditional that could be a guard clause, a database query that would not scale past a thousand rows.

Does it replace code review? No. Does it mean the code that arrives at code review is already past the first pass of obvious issues? Yes.

`/batch` is the bigger deal. Interactive planning followed by parallel execution across dozens of agents, each in its own worktree, each testing and creating a PR independently.

```
> /batch migrate src/ from Solid to React
```

I have not run a large migration with this yet — we are not doing a Solid-to-React migration. But I ran it on a smaller problem: converting our logging calls from a deprecated internal format to our new structured logger. Fourteen files. Six parallel agents. Each tested its own changes. Each opened a PR. Total wall time: eleven minutes. I reviewed and merged five of the six without changes. The sixth had a genuine error in an edge case with conditional logging — the kind of thing that required human judgment.

That ratio matters. Five out of six machine-reviewed migrations is not perfect. It is not meant to be. It is meant to reduce the bottleneck at the point where human attention is most valuable.

Your CLAUDE.md Is Probably Wrong: 7 Mistakes Boris Cherny Never Makes

Boris Cherny's file is 2.5k tokens. Mine was 15k. After 3 weeks studying his workflow, I found the patterns most…

alirezarezvani.medium.com

The Design Philosophy, Assembled

When you read all five parts together, the pattern is visible.

Boris started with the foundation: parallelism, planning, accumulated knowledge.

Then he added the team layer: shared CLAUDE.md, team hooks, checked-in permissions. Then the customization layer: output styles, custom agents, sandboxing. Then native isolation. Then the compound operations that only become possible once everything else is in place.

Each part presupposes the previous one. `/batch` does not make sense without worktrees. Worktrees do not make sense without shared CLAUDE.md. Shared CLAUDE.md does not make sense without plan mode producing consistent, reviewable output.

> *The tips are not a menu. They are a stack.*

Boris said something in his January thread that I keep returning to: *"There is no one correct way to use Claude Code."* I initially read that as permission to pick and choose.

I now read it differently. He means the *order* and *combination* vary by context. The underlying architecture does not.

What I Would Have Done Differently

If I were starting over with this skill today, I would implement it in the order Boris released it. Not because the January tips are more important, but because the February tips build on muscle memory you develop in January.

You cannot configure a useful custom agent until you have burned through enough sessions to know what that agent should constrain. You cannot run `/batch` intelligently until you have failed at enough parallel worktree sessions to understand where human checkpoints belong.

The developers I have seen struggle with Boris's setup are the ones who started with the advanced features. They installed five plugins, configured sandboxing, and set up a `/batch` migration on week one. The cognitive overhead overwhelmed the productivity gain and they went back to single-session Claude Code.

The skill, used in sequence, is a twelve-week ramp. Not because the features are complex, but because the judgment about *when* to use them requires the baseline experience to develop first.

Honest Concerns

The Opus cost adds up. Boris uses Opus 4.5 with thinking for everything because at his throughput, the lower steering overhead makes the math work. At our scale — a team of 7 on a healthtech startup budget — I route monitoring and simple tasks through Sonnet and reserve Opus for architecture decisions and complex debugging. The skill does not address cost routing. You will need to figure that out for your context.

`/simplify` is not a code review replacement. It catches structural issues reliably. It misses domain-specific concerns — whether this validation matches what our API contract specifies, whether this database call respects our read replica routing. Human review remains necessary. The value is that human review now starts at a higher baseline.

The migration to worktrees from checkouts has friction. If your team is already using Boris's January setup with multiple git checkouts, the native worktree flag changes the workflow enough that it needs deliberate migration. We spent half a day aligning on conventions before running our first parallel agent jobs.

Spinner verbs are genuinely cosmetic. I said the Part 3 features were not cosmetic. I stand by that for output styles, custom agents, and sandboxing. The spinner verbs and keybindings customization are cosmetic. They are worth five minutes of configuration and then no further thought. Include them in your `settings.json` because they are there and it is easy. Do not spend time on them beyond that.

The Skill as an Artifact

There is something interesting about the `boris` skill that goes beyond its content.

Boris did not publish a book or a course. He published five threads over eight weeks. Carolina Cherry turned them into an installable skill that Claude Code can update, verify, and self-reference. When you ask Claude Code a workflow question with the skill loaded, it answers against 42 tips from the creator of the tool you are using.

That is a new kind of documentation. Not static. Not passive. Living inside the tool it describes, accessible at the point of need.

I am building something similar for our own team's accumulated Claude Code learnings — the specific patterns that work for our stack, our constraints, our deployment context. The boris skill is the model for how to think about that.

Star the project at howborischernyusesclaudecode.com if the compilation was useful. @CarolinaCherry did the work of turning scattered threads into something usable.

✨ *Subscribe for more production-tested Claude Code patterns and engineering systems content from a CTO building in production.*

About Me

I am Alireza Rezvani (Reza), CTO building AI development systems for engineering teams. I write about turning individual expertise into collective infrastructure through practical automation.

Connect: *Medium* | *Newsletter* | *GitHub*

Claude Code | Claude Code Tips | Agentic Ai | Software Engineering | Agentic Coding

Follow

Written by Reza Rezvani

7.1K followers · 89 following

CTO & AI builder based in Berlin. Writing about Claude Code, agentic workflows, and shipping real products with AI. 20+ years of turning ideas into products.